Learn One Thing at a Time: Curriculum-Based Test-Time Learning for LLM Game Agents

Mohsen Arjmandi*

Abstract

Large language model (LLM) agents deployed in unknown environments must learn task structure at test time, but current approaches require thousands of interactions to form useful hypotheses. We present **Sensi**, an LLM agent architecture for the ARC-AGI-3 game-playing challenge that introduces structured test-time learning through three mechanisms: (1) a two-player architecture separating perception from action, (2) a curriculum-based learning system managed by an external state machine, and (3) a database-as-control-plane that makes the agent's context window programmatically steerable. We further introduce an LLM-as-judge component with dynamically generated evaluation rubrics to determine when the agent has learned enough about one topic to advance to the next. We report results across two iterations: Sensi v1 solves 2 game levels using the two-player architecture alone, while Sensi v2 adds curriculum learning and solves 0 levels—but completes its entire learning curriculum in approximately 32 action attempts, achieving 50–94× greater sample efficiency than comparable systems that require 1,600–3,000 attempts. We precisely diagnose the failure mode as a self-consistent hallucination cascade originating in the perception layer, demonstrating that the architectural bottleneck has shifted from *learning efficiency* to *perceptual grounding*—a more tractable problem.

1 Introduction

The ARC-AGI-3 challenge [1] presents a distinctive test of machine intelligence: agents must play pixel-art puzzle games with no prior knowledge of the rules, mechanics, or win conditions. Unlike standard game-playing benchmarks where the rules are specified or can be inferred from a large training corpus, ARC-AGI-3 requires agents to discover the game's dynamics entirely through interaction—a form of test-time learning that mirrors how humans approach unfamiliar situations.

Current approaches to this challenge are remarkably sample-inefficient. Systems such as Agentica from Symbolica reportedly require 1,600 to 3,000 game interactions to build a working model of a single game's mechanics. This inefficiency fundamentally limits the practical applicability of LLM agents in settings where test-time compute budgets are constrained, a concern that has become increasingly relevant as the field explores test-time scaling [15, 14, 19].

We introduce **Sensi**, an LLM agent that evolves across two design iterations to address this sample efficiency problem. The core architectural insight is separation of concerns: perception should be decoupled from action, and learning should be structured as a curriculum rather than an undirected search.

Sensi v1 splits the agent into two communicating roles—an Observer that maintains hypotheses about the game world, and an Actor that selects actions to test those hypotheses. This two-player architecture, where cognition is factored into perception and decision-making, enables structured

*Correspondence: mohsen.arjmandi@gmail.com. Blog: https://freddiespirit.substack.com (Freddie Spirit).

hypothesis accumulation. Using ChatGPT 5.1 as the backbone, v1 solves 2 levels of the ARC-AGI-3 challenge with the notable property that pass@10 equals pass@1: results are consistently reproducible given the same accumulated knowledge.

Sensi v2 adds three mechanisms on top of the two-player foundation: (i) a *curriculum* that orders learning goals into a sequential queue with a state machine managing progression, (ii) a *database-as-control-plane* where the agent's entire cognitive state resides in SQLite tables that are programmatically injected into prompts each turn, and (iii) a *sense scorer* implementing LLM-as-judge [20] with dynamically generated evaluation rubrics. V2 solves 0 levels—an honest negative result—but completes its full learning curriculum in approximately 32 interactions, a 50–94× improvement in sample efficiency.

The failure mode is itself a contribution: we identify a *self-consistent hallucination cascade* where errors in the perception layer (frame differencing) propagate through the hypothesis pipeline, producing internally coherent but factually incorrect game models that receive high scores from the sense evaluator. This precisely localizes the bottleneck: the architecture works; the perception does not.

Our contributions are:

1. A **two-player architecture** that separates perception from action via distinct Observer and Actor LLM roles communicating through structured hypothesis lists (§3).

2. A **curriculum learning system** with an external state machine that sequences learning goals and accumulates verified knowledge as facts for subsequent learning items (§4).

3. A **database-as-control-plane** pattern that makes the LLM's context window externally programmable through database state, enabling modular and steerable agent behavior (§4.4).

4. An **LLM-as-judge with dynamic rubrics** for evaluating learning progress, where both the evaluation metric and the scoring are performed by separate LLM calls (§4.3).

5. A **sample efficiency analysis** demonstrating 50–94× improvement over baselines, together with a precise diagnosis of the hallucination cascade failure mode (§6).

2 Preliminaries

2.1 ARC-AGI-3 Environment

ARC-AGI-3 [1] is a game-based evaluation where agents play pixel-art puzzle games without prior knowledge of the rules. At each timestep t, the agent observes a game frame represented as a 3D tensor:

$$\mathcal{F}_t \in \mathbb{Z}_{16}^{L \times H \times W} \tag{1}$$

where L is the number of grid layers, H and W are the spatial dimensions (up to 64×64), and each cell takes an integer value in $[0, 15]$ representing a color. The agent selects from a discrete action space:

$$\mathcal{A} = \{a_0, a_1, \ldots, a_7\} \tag{2}$$

where a_0 is RESET (restarts the game), a_1 through a_5 are simple actions (directional movement, interact), a_6 is a coordinate-parameterized action taking (x, y) arguments, and a_7 is an additional simple action. After executing action $a_t \in \mathcal{A}$, the agent receives the next frame $\mathcal{F}_{t+1}$ and a scalar score $s_t \in [0, 254]$, along with a status code from $\{\texttt{NOT_PLAYED}, \texttt{NOT_FINISHED}, \texttt{GAME_OVER}, \texttt{WIN}\}$.

Crucially, the semantics of each action are *not* provided to the agent. The agent must discover what each action does, how the game mechanics work, and what constitutes winning—all from raw interaction.

2.2 POMDP Formulation

The ARC-AGI-3 problem can be formulated as a partially observable Markov decision process (POMDP) [4] with an important distinction: the agent must discover not only the state dynamics but also the reward function itself. Let $\langle \mathcal{S}, \mathcal{A}, T, R, \Omega, O \rangle$ denote the POMDP, where $\mathcal{S}$ is the (hidden) game state space, $\mathcal{A}$ is the action space, $T : \mathcal{S} \times \mathcal{A} \to \Delta(\mathcal{S})$ is the unknown transition function, $R : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is the unknown reward function, Ω is the observation space (rendered frames), and $O : \mathcal{S} \to \Delta(\Omega)$ is the observation function.

The agent's task is two-fold: (1) learn an approximate model of T and R from interactions, and (2) use this model to select actions that maximize the game score. This dual-discovery requirement distinguishes ARC-AGI-3 from standard POMDP settings where the dynamics are assumed known or learnable from a fixed distribution.

2.3 Test-Time Learning

It is important to situate Sensi within the landscape of test-time adaptation methods. We distinguish three paradigms:

Test-time compute (TTC). Methods such as chain-of-thought prompting [15] and self-consistency [14] allocate additional inference-time computation to improve output quality. The model's parameters remain frozen; only the reasoning trajectory is extended.

Test-time training (TTT). Methods such as TTT [11] and TTT-Discover [19] perform gradient-based updates to the model's parameters on each test instance. The model literally learns—in the weight-update sense—at test time.

Test-time in-context learning. Sensi occupies a third position: the model's parameters remain frozen, but the agent accumulates structured knowledge in an external database that is injected into the context window each turn. This is neither pure inference (the agent's effective knowledge base changes) nor gradient-based training (no parameters are updated). We call this *in-context knowledge accumulation with persistent structured state*. The key distinction from standard in-context learning is that the accumulated knowledge persists across context windows through the database, enabling learning trajectories that span hundreds of turns without being limited by context length.

3 Sensi v1: Two-Player Architecture

The foundational insight of Sensi is that a single LLM call asked to simultaneously perceive a game state, reason about what changed, maintain hypotheses, and select an action performs poorly at all four tasks. V1 addresses this by splitting cognition into two cooperating roles.

3.1 Observer and Actor

Each turn in Sensi v1 involves two LLM calls:

$$\text{Player}_1(\mathcal{F}_t, \mathcal{F}_{t-1}, a_{t-1}, \Delta_t) \;\to\; (G_t, K_t) \tag{3}$$

$$\text{Player}_2(G_t, K_t) \;\to\; (d_t, a_t) \tag{4}$$

where G_t is the *guesses* list (hypothesized game mechanics), K_t is the *figured-out* list (confirmed observations), $d_t \in \{\text{GUESS}, \text{INFORMED}\}$ is the decision type indicating whether the action is exploratory or knowledge-based, and Δ_t is a visual description of changes between consecutive frames.

The prompt frames this as cooperative play: "you and your friend are playing a game together where you see the board and your friend chooses an action." This framing leverages the LLM's capacity for perspective-taking and collaborative reasoning.

3.2 Stochastic Exploration via Confidence-Modulated Agency

A distinctive aspect of v1's design is its treatment of *confidence as a dynamic variable* that modulates the agent's exploration behavior. Rather than using fixed exploration schedules (as in ϵ-greedy strategies) or curiosity-driven bonuses [7, 9], Sensi instructs the LLM to act based on what has been figured out—allowing the model to dynamically modulate its behavior based on its evolving epistemic state.

This design exploits an underappreciated capacity of LLMs: the ability to model *stochastic aspects of human agency*. When humans face genuinely unknown situations, their behavior is not purely logical—it includes intuitive leaps, confidence-weighted guesses, and exploratory randomness that collectively help navigate unfamiliar territory. By representing confidence dynamically in the agent's state (through the distinction between guesses and figured-out items), Sensi allows the LLM to approximate this stochastic exploration naturally, without requiring explicit exploration mechanisms.

Formally, the agent's update rule can be expressed as:

$$\text{Sense}(t+1) = \text{Sense}(t) + \text{LLM}(\text{Sense}(t), S(t+1)) \tag{5}$$

$$S(t+1) \sim P(\cdot \mid S(t), A(t)) \tag{6}$$

$$A(t+1) = \text{LLM}(\text{Sense}(t+1)) \tag{7}$$

where $\text{Sense}(t) = (G_t, K_t)$ is the evolving epistemic state. Unlike thinking models that generate stochastic exploration through random token sampling, Sensi's exploration emerges from the *structure of the epistemic state*: when the guesses list is large relative to the figured-out list, the Actor naturally behaves more exploratorily; as items migrate from guesses to figured-out, behavior becomes more directed.

3.3 Results and Limitations

Sensi v1 was evaluated on ARC-AGI-3 using ChatGPT 5.1 as the backbone model. It successfully solved the first 2 levels of game LS20, discovering 15 correct facts about the game mechanics including player movement rules, energy consumption, key-door interactions, and the win condition.

A notable property of v1's results is that **pass@10** = **pass@1**: given the same figured-out list, the Actor consistently selects the correct action sequence. This contrasts with thinking-model approaches where results vary across runs due to sampling stochasticity, and suggests that the two-player architecture produces more deterministic and reproducible behavior.

However, v1 has fundamental limitations: (i) there is no mechanism to control *what* the agent learns or in what order, (ii) there is no way to verify whether the agent has actually learned something correctly, and (iii) at level 3, the agent entered a degenerate state where it stopped updating its understanding and repeated single actions across 100+ turns. These limitations motivate the structured learning approach of v2.

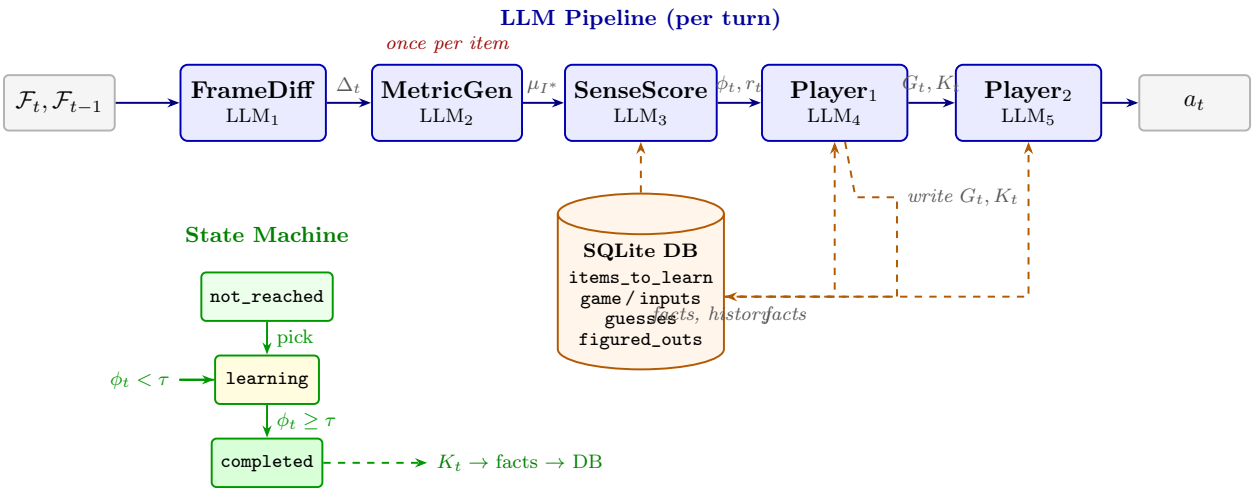

Figure 1: **Sensi v2 architecture.** Each turn involves up to five LLM calls orchestrated through a pipeline. The SQLite database serves as the control plane: all agent state resides in database tables that are queried to construct prompts and updated with each turn's outputs. The curriculum state machine (left) manages learning progression, promoting figured-out items to facts when a learning item is completed. MetricGen (LLM$_2$) runs only once when a new learning item is activated.

4 Sensi v2: Curriculum Learning with Programmable Context

The core idea behind v2 is to impose structure on the learning process itself: **learn one thing at a time**. Instead of allowing the agent to explore an unstructured hypothesis space, v2 introduces a queue of learning items that the agent must master sequentially, with an external state machine managing progression and an LLM-based judge verifying learning outcomes.

4.1 Turn Architecture

Each turn in Sensi v2 involves up to five LLM calls organized as a pipeline:

$$\underbrace{\text{FrameDiff}}_{\text{LLM}_1} \to \underbrace{\text{MetricGen}}_{\text{LLM}_2} \to \underbrace{\text{SenseScore}}_{\text{LLM}_3} \to \underbrace{\text{Player}_1}_{\text{LLM}_4} \to \underbrace{\text{Player}_2}_{\text{LLM}_5} \tag{8}$$

Frame Differencing (LLM$_1$). The raw grid $\mathcal{F}_t$ is rendered to a scaled pixel image via a color map ($\mathbb{Z}_{16} \to$ RGB). Two consecutive frames are passed to a multimodal LLM that returns a structured JSON diff:

```python
def frame_diff_finder(self, current_frame, prev_frame) -> str:
    prediction = self.frame_diff_module(
        prev_frame=DSPyImage(url=encode_image(prev_frame)),
        current_frame=DSPyImage(url=encode_image(current_frame)),
    )
    return prediction.diff_json  # structured JSON
```

Listing 1: Frame differencing module.

The diff schema captures added, removed, and moved objects, UI changes, and a high-level summary—giving the Observer a structured description of *what changed* rather than forcing it to compare raw pixels.

Metric Generation (LLM_2). Called once per learning item when it enters the `learning` state. Generates an evaluation rubric for the sense scorer (detailed in §4.3).

Sense Scoring (LLM_3). Called every turn for the active learning item. Produces a score $\phi_t \in [1, 10]$ and reasoning text (detailed in §4.3).

Player 1—Observer (LLM_4). Receives the game frame, frame diff, previous guesses and figured-out lists, accumulated facts, the current learning target, sense score feedback, and action history. Outputs updated guesses and figured-out lists. The prompt includes guidelines on list curation: keep valid items, edit with new information, remove contradicted items, and promote confirmed guesses to figured-out.

Player 2—Actor (LLM_5). Receives guesses, figured-out lists, facts, and the current learning target. Outputs a decision type ($d_t \in \{\text{GUESS}, \text{INFORMED}\}$) and exactly one action. The Actor's objective is to resolve the Observer's uncertainties by choosing actions that help discriminate between competing hypotheses.

4.2 Learning Queue and State Machine

At the start of each game, Sensi v2 initializes a learning curriculum stored in an SQLite table:

```python
def initialize_items_to_learn(self, game_id, card_id):
    # Facts: already-known truths
    fact_items = [
        "RESET starts the game",
        "all available actions: ACTION1, ..., ACTION7, RESET",
    ]
    # Learning items: to be mastered sequentially
    default_items = [
        "learn what each action does in the game",
        "learn how actions affects your energy while playing",
        "learn how to win the game",
    ]
    for item_name in default_items:
        cur.execute("""
            INSERT OR IGNORE INTO items_to_learn
            (game_id, card_id, item_name, state, threshold)
            VALUES (?, ?, ?, 'not_reached', 8)
        """, (game_id, card_id, item_name))
```

Listing 2: Curriculum initialization.

Each learning item follows a three-state lifecycle managed by a state machine:

Formally, let $\mathcal{I} = \{I_1, I_2, \ldots, I_n\}$ be the ordered sequence of learning items, each with state $\sigma_i \in \{\texttt{not_reached}, \texttt{learning}, \texttt{completed}, \texttt{fact}\}$. At any turn t, the *active item* is:

$$I^* = \min_i \{I_i \mid \sigma_i \in \{\texttt{learning}, \texttt{not_reached}\}\} \tag{9}$$

When I^* transitions to `completed`, its accumulated figured-out entries are promoted to facts:

$$\sigma_{I^*} = \texttt{completed} \implies \forall\, k \in K_t : \sigma_k \leftarrow \texttt{fact} \tag{10}$$

These new facts become part of the context for subsequent learning items, creating a **knowledge accumulation chain**: each learning item builds upon the verified (or putatively verified) knowledge from all previous items.

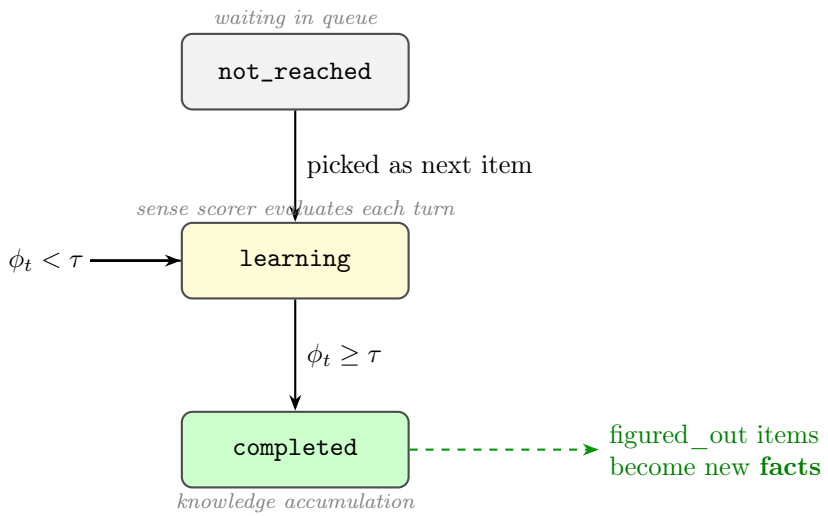

Figure 2: **Learning item state machine.** Each curriculum item progresses through three states. The self-loop on `learning` represents repeated evaluation until the sense score meets the threshold τ. Upon completion, accumulated figured-out items are promoted to facts, creating a knowledge accumulation chain.

4.3 Sense Scoring: LLM-as-Judge for Learning Progress

How does the agent know when it has learned enough about a topic? V2 introduces a *sense scorer*—a separate LLM call acting as an external judge. Critically, the judge does not use a fixed rubric. Instead, the rubric itself is dynamically generated by another LLM call, creating a two-phase evaluation system.

Phase 1: Metric Generation. When a new learning item becomes active, a `MetricGeneratorSignature` is invoked to produce a verification criterion:

```
class MetricGeneratorSignature(dspy.Signature):
    """Given an item the agent needs to learn about a game,
    generate a metric to verify that the item has been learned."""

    item_to_learn: str = dspy.InputField(
        desc="The item/concept the agent needs to learn"
    )
    learning_metric: str = dspy.OutputField(
        desc="A criteria and description that a judge will use
         to give a score on how good a grasp the learner has
         on the item. Score between 1 to 10."
    )
```

Listing 3: Dynamic metric generation via DSPy.

Table 1: **Database schema.** Six SQLite tables constitute the agent's externalized cognitive state.

Table	Purpose
`items_to_learn`	Learning curriculum with state machine
`inputs`	Key-value store for per-turn game state
`game`	Turn-by-turn history of frames, actions, diffs
`guesses`	Player 1's hypothesis lists per turn
`figured_outs`	Player 1's confirmed observations per turn
`losing_action_seqs`	Action sequences that led to game over

Phase 2: Sense Scoring. On each subsequent turn, the `SenseScorerSignature` evaluates the agent's current understanding against the generated metric:

```
class SenseScorerSignature(dspy.Signature):
    """Score the agent's understanding of a learning item."""

    item_to_learn: str = dspy.InputField()
    learning_metric: str = dspy.InputField()
    facts: List[str] = dspy.InputField()
    figured_out: List[str] = dspy.InputField()

    sense_score: int = dspy.OutputField(
        desc="Score from 1-10 indicating learning progress."
    )
    reasoning: str = dspy.OutputField(
        desc="Brief explanation for the score"
    )
```

Listing 4: Sense scoring via DSPy.

Let μ_i denote the metric generated for item I_i, and ϕ_t the sense score at turn t. The transition condition is:

$$\phi_t(I^*, \mu_{I^*}, \mathcal{K}_t, K_t) \;\geq\; \tau_{I^*} \quad \Longrightarrow \quad \sigma_{I^*} \leftarrow \texttt{completed} \tag{11}$$

where $\mathcal{K}_t$ is the accumulated fact set and τ_{I^*} is the threshold (default $\tau = 8$ out of 10). The sense reasoning r_t is fed back to Player 1 on the next turn, providing explicit feedback about *why* the score was what it was and what knowledge is still missing.

4.4 Database as Control Plane

One of the most architecturally significant choices in v2 is using an SQLite database as the agent's *programmable context*. Every turn, the agent reads its state from and writes its state to six database tables (Table 1).

This is not mere persistence—it is a **control plane**. By modifying database contents, one changes the agent's behavior without altering code or prompts. Want the agent to skip action learning? Delete those rows. Want to seed it with human-provided facts? Insert them. Want to change the learning order? Reorder the items.

The LLM's context window becomes programmable through database state:

$$\text{Prompt}_t = f(\mathcal{F}_t,\ \text{DB.}\texttt{facts},\ \text{DB.}\texttt{item_to_learn},\ \text{DB.}\texttt{history},\ \text{DB.}\texttt{sense_score}) \tag{12}$$

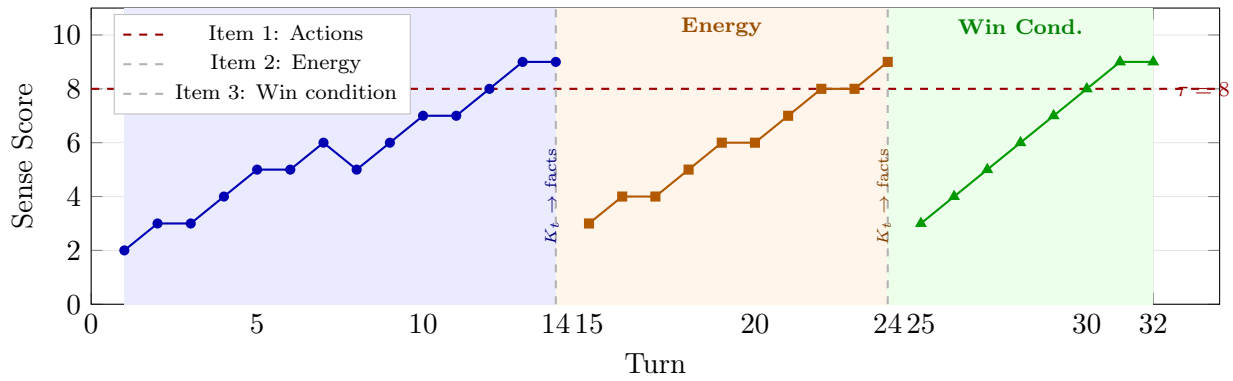

Figure 3: **Sense score progression over turns** (illustrative). The agent's sense score for each curriculum item rises toward the threshold $\tau = 8$ as it accumulates figured-out items. Vertical dashed lines mark curriculum transitions where one item is completed and the next begins. The entire curriculum is completed in approximately 32 turns. Scores are representative of observed behavior; exact per-turn values vary across runs.

This architecture differs fundamentally from standard prompt engineering or template-based agent frameworks. In those approaches, the prompt structure is static and the content varies only through direct substitution. In Sensi v2, the database mediates between the symbolic control structure (curriculum, state machine) and the neural inference engine (LLM), creating a form of *neuro-symbolic programming* where the symbolic layer guides the neural layer through structured learning.

The practical implication is *external steerability*: the entire learning process can be monitored, modified, and debugged through standard database operations. This is analogous to how control planes in distributed systems separate the data path (what the system does) from the control path (how the system is configured).

4.5 POMDP with Internal Reward

We can frame Sensi v2's learning process as a POMDP augmented with an internal reward signal. Define the agent's epistemic state at time t as:

$$\mathcal{E}_t = \big(\mathcal{K}_t,\ G_t,\ K_t,\ I_t^*,\ \phi_t,\ \mu_{I_t^*}\big) \tag{13}$$

where $\mathcal{K}_t$ is the fact set, G_t is guesses, K_t is figured-out, I_t^* is the active learning item, ϕ_t is the sense score, and $\mu_{I_t^*}$ is the evaluation metric for the active item. The transition dynamics are:

$$\mathcal{E}_{t+1} = T(\mathcal{E}_t,\ a_t,\ \mathcal{F}_{t+1},\ \mathrm{LLM}_1(\mathcal{F}_t, \mathcal{F}_{t+1}),\ \mathrm{LLM}_3(\cdot),\ \mathrm{LLM}_4(\cdot)) \tag{14}$$

The policy π is the composition of all five LLM modules:

$$a_t = \pi(\mathcal{E}_t, \mathcal{F}_t) = \mathrm{LLM}_5\Big(\mathrm{LLM}_4\big(\mathcal{F}_t, \Delta_t, \mathcal{E}_t\big)\Big) \tag{15}$$

What distinguishes this formulation is that the **reward signal is internal**. The agent does not directly optimize for the game score. Instead, it optimizes for the sense score ϕ_t on the current learning item. The game score is an emergent property of learning the correct things in the correct order. This is analogous to intrinsic motivation in reinforcement learning [9, 7], but implemented through LLM-based self-evaluation rather than curiosity-driven exploration bonuses [12].

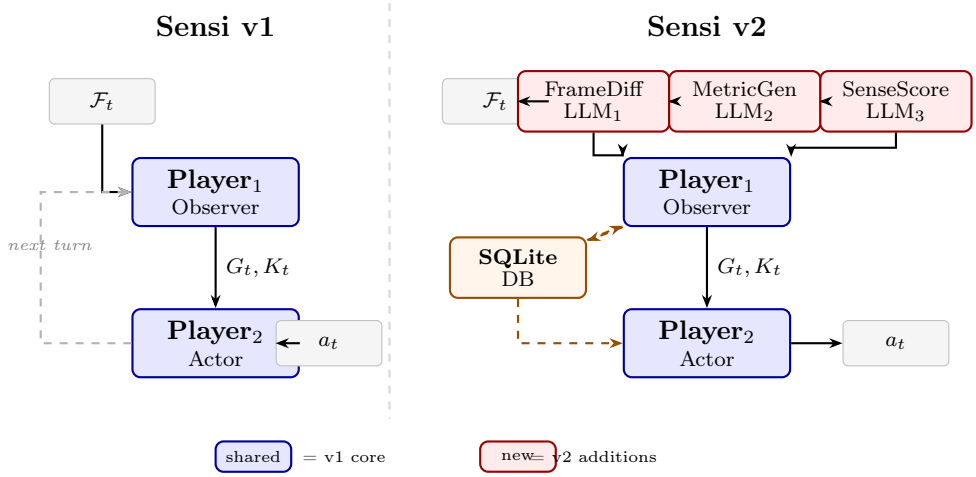

Figure 4: **Architectural comparison of Sensi v1 and v2.** V1 (left) uses a simple two-player loop with shared hypothesis lists. V2 (right) preserves the Player$_1$/Player$_2$ core (blue) but adds frame differencing, dynamic metric generation, sense scoring, and the database-as-control-plane (red/orange). The two-player core is embedded within a richer learning infrastructure.

5 Experiments

5.1 Setup

Sensi v1 uses ChatGPT 5.1 (Thinking High) as the backbone model for both Observer and Actor roles. **Sensi v2** uses Gemini 3.1 Pro for all five pipeline stages. Both versions are implemented using DSPy [5] for structured LLM interactions, with SQLite for state persistence. The sense score threshold is $\tau = 8/10$ for all learning items.

We note that the use of different backbone models across versions is a confounding factor. This was driven by practical considerations (model availability and multimodal capabilities at time of development) rather than experimental design. We discuss this limitation in §9.

5.2 Baselines

We compare against the following:

- **Sensi v1**: The two-player architecture without curriculum learning.
- **Agentica (Symbolica)**: A reported system requiring approximately 1,600–3,000 game interactions per game to build understanding.
- **Random agent**: Uniform random action selection as a lower bound.

Our primary comparison axis is *sample efficiency*—the number of game interactions required to complete an exploration or learning phase—rather than win rate alone. This is because sample efficiency determines the practical viability of test-time learning under compute constraints.

Table 2: **Comparison of Sensi versions and baselines.** Sample efficiency is measured as the number of game interactions to complete a learning or exploration phase. V2 achieves dramatically higher sample efficiency despite solving fewer levels.

System	Backbone	Levels Won	Tries	Curriculum	LLM Judge
Random agent	—	0	—	×	×
Agentica (Symbolica)	Unknown	Unknown	1,600–3,000	×	×
Sensi v1	ChatGPT 5.1	2	Variable	×	×
Sensi v2	Gemini 3.1 Pro	0	∼32	✓	✓

6 Results and Analysis

6.1 Task Performance

We report task performance honestly and upfront: **Sensi v1 solves 2 levels; Sensi v2 solves 0 levels.** Table 2 summarizes the comparison.

The v2 result—0 levels won—deserves contextualization. The agent *did* complete its entire learning curriculum. It systematically explored what each action does, learned about the energy system, and formed a hypothesis about the win condition. The learning pipeline executed correctly: the state machine transitioned as designed, sense scores rose to meet thresholds, and figured-out items were promoted to facts. The agent learned the wrong things, but it learned them efficiently and systematically.

6.2 Curriculum Execution

The curriculum state machine operated correctly across all runs. The agent:

1. Started with the first learning item ("learn what each action does"),
2. Accumulated guesses and figured-out items about action effects,
3. Received rising sense scores as the figured-out list grew,
4. Transitioned to `completed` when $\phi_t \geq \tau = 8$,
5. Promoted figured-out items to facts,
6. Advanced to the next learning item ("learn about energy"),
7. Repeated the cycle through all curriculum items.

This validates the programmable context architecture: the database-mediated curriculum successfully guided the LLM agent through a structured learning process spanning multiple game resets. The agent's behavior changed appropriately at each curriculum transition, demonstrating that external symbolic control can effectively steer neural inference.

6.3 Sample Efficiency

The agent completed its entire learning curriculum in approximately **32 action attempts**. For comparison, Agentica from Symbolica reportedly requires 1,600 to 3,000 interactions per game:

$$\text{Sample efficiency ratio} = \frac{N_{\text{baseline}}}{N_{\text{Sensi}}} = \frac{1600\text{–}3000}{32} \approx 50\text{–}94\times \tag{16}$$

This 50–94× improvement in sample efficiency is the paper's central quantitative result. Even though v2 learned incorrect facts, the *speed* of learning is what matters for the broader research agenda. If an architecture can learn from 32 interactions, the bottleneck is not sample efficiency but *grounding*—ensuring that what the LLM perceives is actually correct. This is a fundamentally different (and more tractable) problem.

In the context of test-time compute research [19], this is especially relevant. Test-time compute is expensive. An architecture that needs 32 interactions to form hypotheses about a new task is fundamentally more practical than one requiring 3,000, particularly when combined with budget-constrained inference pipelines.

6.4 Failure Analysis: Self-Consistent Hallucination Cascade

The failure mode is instructive and constitutes a diagnostic contribution. The frame differencing step (LLM_1) produces natural language descriptions of visual changes between frames. Gemini 3.1 Pro, when examining small pixel-art frames scaled to 10×, frequently misidentifies objects, positions, and movements. A player moving left might be described as "object shifted right." A UI element showing remaining energy might be described as "decorative border pattern."

These errors propagate through the entire pipeline in what we term a **self-consistent hallucination cascade**:

$$\Delta_t^{\text{wrong}} \;\rightarrow\; K_t^{\text{wrong}} \;\rightarrow\; \phi_t^{\text{high}} \;\rightarrow\; \sigma_{I^*} = \texttt{completed} \;\rightarrow\; \mathcal{K}_{t+1}^{\text{wrong}} \tag{17}$$

The cascade operates as follows:

1. **Perception error.** The frame differencing LLM (LLM_1) produces an incorrect diff Δ_t^{wrong}.
2. **Hypothesis contamination.** Player 1 (LLM_4) builds figured-out items based on the wrong diff, producing K_t^{wrong}.
3. **Spurious validation.** The sense scorer (LLM_3) evaluates the agent's understanding based on the *internal consistency* of the figured-out items—not their correspondence to ground truth. Since the items are coherent (just wrong), the scorer assigns a high score ϕ_t^{high}.
4. **Premature completion.** The high score triggers the state machine transition: $\sigma_{I^*} \leftarrow$ `completed`.
5. **Error compounding.** The (wrong) figured-out items are promoted to facts $\mathcal{K}_{t+1}^{\text{wrong}}$, which become part of the context for the next learning item, compounding the error.

This diagnosis is precise and actionable. The architecture—the curriculum, the state machine, the sense scoring pipeline—all function correctly. The information flow is exactly right. The perception layer is the single point of failure, and it is the most straightforward component to improve through better vision models, hybrid programmatic-neural pixel analysis, or ground-truth anchoring mechanisms.

7 Discussion

Architecture validated independently of task performance. The 0-level result for v2 might appear to invalidate the approach, but we argue the opposite. The curriculum executed correctly, the state machine transitioned properly, the sense scores rose and fell as expected, and the agent

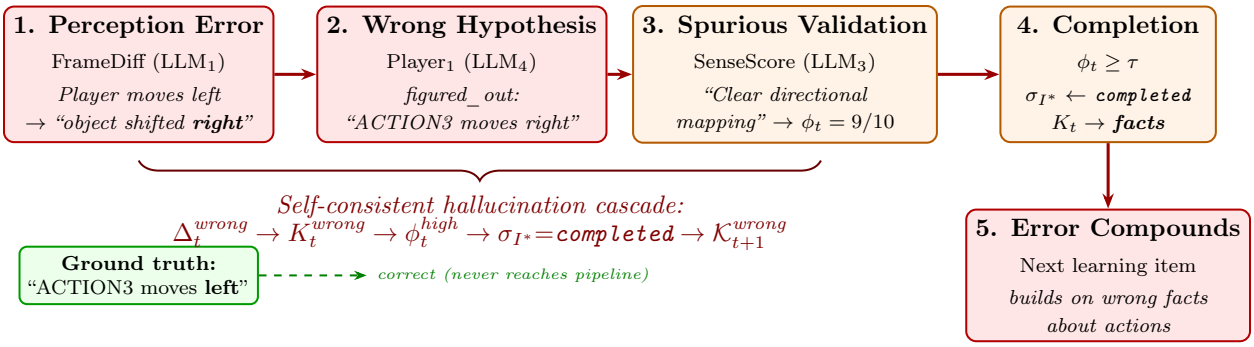

Figure 5: **Self-consistent hallucination cascade.** A frame differencing error (step 1) propagates through the pipeline. Player$_1$ builds a wrong hypothesis (step 2), which the sense scorer validates because it is *internally consistent* (step 3). This triggers premature completion (step 4), and the wrong knowledge contaminates subsequent learning items (step 5). The ground truth (green, bottom-left) never enters the pipeline.

systematically worked through its learning queue. The failure is localized to a single, identifiable component (frame differencing), not to the architectural design. This is analogous to validating a sorting algorithm's correctness even when the comparator function is buggy: the algorithm works; the comparator needs fixing.

The bottleneck has shifted. Before Sensi v2, the question was "can LLM agents learn game mechanics efficiently?" After v2, the question becomes "given that they can learn efficiently, how do we ensure they learn correctly?" This is a more tractable problem because perceptual grounding admits direct engineering solutions: programmatic pixel comparison, fine-tuned vision models, structured visual analysis, or hybrid approaches that combine LLM perception with deterministic pixel differencing. The curriculum and learning architecture, by contrast, required novel design.

Implications for test-time compute budgets. The 32-try result has practical implications beyond games. Any test-time inference scenario—novel tool use, unfamiliar APIs, new document formats—could benefit from architectures that form structured hypotheses from minimal interactions. A 50–94× reduction in required interactions translates directly to reduced cost and latency in deployment settings where test-time compute is budgeted.

Database-as-control-plane as a general pattern. We believe the database-as-control-plane pattern is underexplored in the agent architecture literature. Most agent frameworks manage context through prompt templating—static templates with variable substitution. Sensi v2 demonstrates that externalizing cognitive state to a database and programmatically injecting it into prompts creates more modular, steerable, and debuggable agents. This pattern naturally supports multi-agent coordination, human-in-the-loop oversight, and post-hoc analysis of agent behavior through standard database queries.

8 Related Work

Test-time computation and training. The spectrum of test-time adaptation ranges from inference-time reasoning [15, 14] through in-context learning to full gradient-based test-time training [11, 19]. TTT-Discover [19] is most relevant to our work: it performs reinforcement learning on a single test problem, training the model's weights to discover solutions. Sensi differs fundamentally in that no gradient updates occur; instead, learning is realized through structured knowledge

accumulation in an external database. This makes Sensi applicable to black-box LLM APIs where weight access is unavailable.

LLM agents for games and environments. Voyager [13] builds an open-ended embodied agent with a skill library, curriculum, and code-as-action paradigm in Minecraft. Unlike Sensi, Voyager operates in a well-understood environment with known mechanics. ReAct [18] interleaves reasoning and acting but does not maintain persistent state across episodes. Reflexion [10] adds verbal reinforcement learning through self-reflection, which is conceptually similar to Sensi's sense scoring but without the curriculum structure or external state machine. SPRING [16] uses game manuals as grounding documents—the opposite of Sensi's approach, which assumes no prior game knowledge.

Curriculum learning. The idea of ordering training examples by difficulty dates to Bengio et al. [2] and has been extensively studied in deep RL [8]. Sensi's contribution is applying curriculum learning at *test time* to structure an LLM agent's exploration, rather than ordering training data during model development. The curriculum is hand-designed in the current work; automatic curriculum generation [8] is a natural extension.

LLM self-evaluation. The use of LLMs as judges has been studied extensively [20], primarily for evaluating model outputs in benchmarking contexts. Self-Refine [6] uses self-feedback for iterative improvement. Sensi's sense scorer extends this paradigm in two ways: (1) the evaluation rubric is itself generated by an LLM, making the system adaptive to arbitrary learning domains, and (2) the score is used as a control signal for a state machine rather than just feedback for refinement.

ARC and abstraction reasoning. The ARC benchmark [3] tests abstract reasoning through visual pattern completion. Program synthesis approaches [17] attempt to solve ARC tasks by generating programs. ARC-AGI-3 [1] extends this to interactive game-playing, requiring online learning rather than one-shot inference. Sensi is, to our knowledge, the first architecture to apply curriculum-based test-time learning specifically to the ARC-AGI-3 game-playing challenge.

9 Limitations and Future Work

Limitations. Several limitations should be noted: (1) Sensi v2 won 0 game levels, meaning the architecture has not yet demonstrated end-to-end task success with curriculum learning. (2) The curriculum is hand-designed; the choice and ordering of learning items requires human judgment about what is learnable and in what order. (3) V1 and v2 use different backbone models (ChatGPT 5.1 and Gemini 3.1 Pro, respectively), confounding direct comparison of architectural changes with model capability differences. (4) The sample efficiency comparison to Agentica/Symbolica is based on reported numbers rather than controlled experiments with identical conditions. (5) The sense scorer evaluates internal consistency rather than ground-truth correctness, creating a fundamental vulnerability to coherent hallucinations.

Future work. The most immediate priority is **fixing the perception layer**. Candidate approaches include: (i) hybrid programmatic and LLM-based frame differencing, where deterministic pixel comparison provides ground truth that the LLM interprets; (ii) ground-truth anchoring for the sense scorer, where a subset of evaluation criteria are verified against actual game state rather than relying solely on LLM judgment; and (iii) fine-tuned vision models specifically trained on pixel-art game frames.

Beyond perception, several directions are promising: **automatic curriculum generation**, where the agent determines its own learning order based on initial exploration; **cross-game transfer**, where facts learned in one game inform hypotheses in structurally similar games; and **multi-agent curriculum learning**, where multiple agents with different curricula share and validate

knowledge through the database-as-control-plane.

10 Conclusion

We presented Sensi, an LLM agent architecture that evolves across two iterations to address the sample efficiency problem in test-time game learning. Sensi v1 demonstrates that separating perception from action via a two-player architecture enables systematic hypothesis accumulation, solving 2 ARC-AGI-3 levels. Sensi v2 adds curriculum learning, a database-as-control-plane, and LLM-as-judge evaluation, achieving 50–94× greater sample efficiency than comparable systems—completing its learning curriculum in approximately 32 interactions versus 1,600–3,000 for baselines.

V2's 0-level result is an honest negative finding, but the failure mode is precisely diagnosed: a self-consistent hallucination cascade originating in the perception layer, where frame differencing errors produce internally coherent but factually incorrect knowledge that passes the sense scorer's consistency checks. This diagnosis shifts the research bottleneck from "how to learn efficiently" to "how to perceive correctly"—a more tractable engineering problem.

The broader contribution is methodological: structured test-time learning—with curriculum, state machine, and programmable context—works as an architectural pattern. The 32-try budget suggests that LLM agents can form structured hypotheses about unknown environments with human-like sample efficiency, provided the perception layer is reliable. Fixing perception is the next step; the learning architecture is ready.

Acknowledgments

The author thanks the ARC Prize Foundation for creating the ARC-AGI-3 challenge. The Sensi agent code is built on the ARC-AGI-3-Agents framework.[1] This work was conducted independently without institutional funding.

References

[1] ARC Prize Foundation. ARC prize 2024: Technical report. 2024. URL https://arcprize.org/blog/arc-prize-2024-technical-report.

[2] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. *Proceedings of the 26th International Conference on Machine Learning*, pages 41–48, 2009.

[3] François Chollet. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019.

[4] Leslie Pack Kaelbling, Michael L Littman, and Anthony R Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1-2):99–134, 1998.

[5] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T Joshi, Hanna Mober, et al. DSPy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2023.

[6] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. 36, 2024.

[1] https://github.com/arcprize/ARC-AGI-3-Agents

[7] Deepak Pathak, Pulkit Agrawal, Alexei A Efros, and Trevor Darrell. Curiosity-driven exploration by self-supervised prediction. *Proceedings of the 34th International Conference on Machine Learning*, pages 2778–2787, 2017.

[8] Rémy Portelas, Cédric Colas, Lilian Weng, Katja Hofmann, and Pierre-Yves Oudeyer. Automatic curriculum learning for deep RL: A short survey. In *International Joint Conference on Artificial Intelligence*, pages 4819–4825, 2020.

[9] Jürgen Schmidhuber. A possibility for implementing curiosity and boredom in model-building neural controllers. *Proceedings of the International Conference on Simulation of Adaptive Behavior*, pages 222–227, 1991.

[10] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2024.

[11] Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Kumbhar, et al. Learning to (learn at test time): RNNs with expressive hidden states. *arXiv preprint arXiv:2407.04620*, 2024.

[12] Richard S Sutton and Andrew G Barto. Reinforcement learning: An introduction. MIT Press, 2018.

[13] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. In *Advances in Neural Information Processing Systems*, volume 36, 2024.

[14] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023.

[15] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837, 2022.

[16] Yue Wu, Yewen Xie, Shrimai Shi, Xiang Yu, Fei Liu, Yuting Liu, Jakob N Foerster, and William Yang Clark. SPRING: Studying the paper and reasoning to play games. *arXiv preprint arXiv:2305.15486*, 2023.

[17] Yudong Xu, Elias B Khalil, and Scott Sanner. Program synthesis for ARC. *arXiv preprint arXiv:2306.03553*, 2023.

[18] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

[19] Mert Yuksekgonul, Daniel Koceja, Xinhao Li, Federico Bianchi, Jed McCaleb, Xiaolong Wang, Jan Kautz, Yejin Choi, James Zou, Carlos Guestrin, and Yu Sun. Learning to discover at test time. *arXiv preprint arXiv:2503.07432*, 2025.

[20] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P Xing, et al. Judging LLM-as-a-Judge with MT-Bench and chatbot arena. 36, 2024.

A Sensi v1: Discovered Game Knowledge

For completeness, we list the 15 figured-out items that Sensi v1 discovered for game LS20, Level 1. These items were accumulated over the course of gameplay and represent the agent's complete model of the game mechanics:

1. RESET starts the game.
2. Blue platform (with red top) is our player.
3. ACTION1 moves the player 1 pixel up.
4. ACTION2 moves the player 1 pixel down.
5. ACTION3 moves the player 1 pixel left.
6. ACTION4 moves the player 1 pixel right.
7. Each action consumes 1 unit of energy shown as dots.
8. Key generator produces matching keys for doors.
9. Walking into a key generator creates a key of that color.
10. Keys are consumed when used to open matching doors.
11. Doors disappear when opened with the correct key.
12. Energy dots provide additional energy when collected.
13. Game over occurs when energy reaches zero.
14. Stars are collectible items.
15. Collecting all stars completes the level.

These items were discovered purely through interaction, without any prior game knowledge. The accuracy of these items (verified against actual game mechanics) demonstrates that the two-player architecture can produce correct game models when the perception layer functions adequately.

B Algorithm: Sensi v2 Turn Execution

Algorithm 1 Sensi v2: Single Turn Execution

Require: Game frame $\mathcal{F}_t$, previous frame $\mathcal{F}_{t-1}$, database DB

Ensure: Action a_t

$\Delta_t \leftarrow \text{FrameDiff}(\mathcal{F}_{t-1}, \mathcal{F}_t)$ ▷ LLM_1: visual differencing

$I^* \leftarrow \text{DB.get_active_item}()$ ▷ Current learning item

$\mathcal{K}_t \leftarrow \text{DB.get_facts}()$ ▷ Accumulated facts

if $\sigma_{I^*} = \texttt{not_reached}$ **then**

  $\sigma_{I^*} \leftarrow \texttt{learning}$

  $\mu_{I^*} \leftarrow \text{MetricGen}(I^*)$ ▷ LLM_2: generate rubric

  $\text{DB.store_metric}(I^*, \mu_{I^*})$

end if

$\phi_t, r_t \leftarrow \text{SenseScore}(I^*, \mu_{I^*}, \mathcal{K}_t, K_{t-1})$ ▷ LLM_3: evaluate

if $\phi_t \geq \tau_{I^*}$ **then**

  $\sigma_{I^*} \leftarrow \texttt{completed}$

  $\text{DB.promote_to_facts}(K_{t-1})$ ▷ Knowledge accumulation

end if

$G_t, K_t \leftarrow \text{Player}_1(\mathcal{F}_t, \Delta_t, \mathcal{K}_t, G_{t-1}, K_{t-1}, I^*, r_t)$ ▷ LLM_4

$\text{DB.store}(G_t, K_t)$

$d_t, a_t \leftarrow \text{Player}_2(G_t, K_t, \mathcal{K}_t, I^*)$ ▷ LLM_5

$\text{DB.log_action}(a_t, d_t)$

return a_t